웹서비스설계 보고서 작성 양식

JWT 게 시 판
구 현 보 고 서

이 양식은 본인이 구현한 결과를 설명하기 위한 문서입니다. 대괄호 안내를 실제 내용으로 바꾸고 작성 안내 문장은 제출 전에 지우세요. 화면 캡처와 표의 행은 필요에 따라 추가할 수 있습니다. 6~10 쪽 내외를 권장하며 분량 자체는 평가하지 않습니다.

01 서 비 스 소 개 와 구 현 범 위

학번 [202117535] 이름: [강동하] 제출일: [2026/10/28]

서비스 이름 [학습 기록 게시판]

기술 조합과 버전 [Vue 3.5.43 (Vite 8.3.4) Express 5.3.0 Node.js v24.19.0 SQLite (sql.js 1.14.2)]

GitHub 저장소 URL [https://github.com/dongha1509-collab/study-log-board]

소스 코드 기준 [브랜치 / 코드 구현 기준 커밋 ID 를 기록할 수 있습니다. 최종 제출 기준 커밋 SHA 는 LMS 에 제출합니다.]

[학습 기록은 공부한 내용을 짧은 글로 남기고 서로 공유하는 게시판으로, 수업과 자기 학습을 정리하려는 학생을 대상으로 합니다. 누구나 로그인 없이 글 목록과 내용을 읽을 수 있고, 로그인한 사용자만 글을 쓰며 자신이 쓴 글만 수정·삭제할 수 있습니다. 수업 시간에 다룬 Vue 와 Express 조합을 이어서 쓰는 것이 구조를 이해하기 쉽다고 판단해 선택했고, 데이터는 SQLite 파일에 저장해 서버를 재시작해도 글이 유지되게끔 했습니다 . 제공 템플릿에서는 인증 관련 파일(auth.js, 프런트의 api.js, App.vue)을 중심으로 수정했고, 게시판 CRUD 와 DB 저장 코드는 그대로 사용했습니다.]

기능	완료 여부	관련 파일	비고 또는 제한
공개 목록과 상세	[완료]	[backend/src/app.js, frontend/src/App.vu]	[제공 코드. 로그인 없이 조회]
로그인과 JWT 발급	[완료]	[backend/src/auth.js issueToken, frontend/src/api.js login]	[토큰 유효 시간 기본 90 초]
인증과 내 정보 조회	[완료]	[backend/src/auth.js currentUserId, backend/src/app.js /api/me]	[변조는 401]
본인 글 작성·수정·삭제	[완료]	[backend/src/app.js, backend/src/auth.js assertOwner, frontend/src/api.js canEdit]	[서버에서 403, 화면에서는 버튼 숨김]
만료·로그아웃·오류 표시	[완료]	[frontend/src/api.js request·clearSession, frontend/src/App.vue run]	[401 이면 로그인 해제, 403 은 유지]
DB 데이터 유지	[완료]	[backend/src/store.js]	[data/board.sqlite 에 저장]

구 조 와 변 경 내 역

브라우저 → API 서버 → SQLite 의 연결 구조를 간단히 그리거나 글로 설명하세요. 로그인 요청과 게시글 수정 요청의 흐름을 구분하고, 토큰이 어디에 저장되고 어느 요청에서 전송되는지 표시하세요.

[화면에서 /api 로 보낸 요청은 Vite 프록시를 거쳐 Express 서버로 가고, 서버가 SQLite 파일에 글을 저장하거나 읽는다.
로그인 요청: 로그인 폼에서 이메일과 비밀번호를 POST /api/auth/login 으로 보낸다. 서버가 비밀번호를 확인하고 맞으면 JWT 를 만들어 accessToken, expiresIn, user 로 보내 준다. 화면은 이 토큰을브라우저 메모리의 session 변수에만 저장한다.
글 수정 요청: PUT /api/posts/{id}를 보낼 때 화면이 Authorization: Bearer <토큰> 헤더를 붙인다. 서버는 먼저 토큰을 확인하고(currentUserId), 글이 있는지 본 뒤, 요청한 사람과 글 작성자가 같은지 비교한다(assertOwner). 같을 때만 DB 를 고친다. 글 목록과 상세 조회는 토큰이 필요 없다.]

변경한 파일과 함수	제공 코드의 원래 역할	내가 변경한 내용	변경 이유
[backend/src/auth.js/ 파일 위쪽 설정 코드]	[없음]	[.env 의 JWT_SECRET 을 읽어 64 자리 16 진수인지 검사하고 32 바이트 키로 바꿈.]	[형식 오류인 키는 서버가 시작되지 않게 하기 위해]
[backend/src/auth.js / issueToken]	[default 로 501 에러]	[HS256 JWT 발급]	[올바른 계정에 토큰줌]
[backend/src/auth.js / currentUserId]	[항상 사용자 1 반환]	[헤더 확인, jwt.verify, exp·sub 확인, 사용자 존재 확인]	[요청마다 토큰을 서버가 검증하기 위해]
backend/src/app.js / 로그인 라우트, import]	[TOKEN_TTL 을 직접 읽음]	[auth.js 의 ttl 사용]	[응답의 expiresIn 과 토큰 exp 를 일치시키기 위해]
[frontend/src/api.js / login]	[에러를 던짐]	[로그인 요청을 보내고 받은 토큰과 사용자를 session 에 저장]	[로그인 상태연결]
[frontend/src/api.js / canEdit]	[항상 true]	[로그인 사용자 id 와 글의 authorId 가 같을 때만]	[남의 글에는 수정·삭제 버튼을 숨기기 위해]
[frontend/src/App.vue / 배너 문구]	[시작코드 미완성 안내]	[로그인한 사용자만 글을 쓸 수 있다]	[안내를 위해]

각 변경의 작성 출처도 구분하세요. 예: 제공 코드 유지, 본인 작성, AI 초안 후 본인 수정, 외부 예제 참고. 보조 파일을 추가했다면 함께 기록하세요.

로 컬 실 행 기 록

실제 설치·빌드·실행 명령과 결과를 적고, 자세한 재현 방법은 저장소 README 에서 연결하세요.

[Windows 11 Pro (10.0.26200), Node.js v24.19.0, npm 11.17.0 / 새 폴더 설치 / 환경설정 / frontend 폴더에서 npm run build 를 실행한다. / backend 폴더에서 npm start 를 실행한다. / backend/data/board.sqlite]

[frontend 에서 npm run preview 를 실행하고 http://127.0.0.1:4173 에 접속했다. 화면에서 로그인하자 login 요청이 200 으로 나왔다./ 서버 재시작 후 데이터 유지는 04 테스트 결과 (T15)에서 확인할 수 있다]

핵 심 코 드 설 명

3 1 로 그 인 과 JWT 발 급

파일 / 함수: [backend/src/auth.js issueToken, backend/src/app.js 로그인 라우트(POST /api/auth/login)]

코 드 발 췌

```
export function issueToken(user) {
  return jwt.sign({}, key, {
    algorithm: "HS256",
    subject: String(user.id),
    issuer,
    audience,
    expiresIn: ttl,
  });
}

const accessToken = issueToken(user);
  res.json({
    accessToken,
    expiresIn: ttl,
    user,
  });
});
```

동 작 과 응 답 코 드 설 명

[입력: 로그인 화면에서 보낸 이메일과 비밀번호
처리: 서버는 먼저 이메일과 비밀번호가 문자열인지, 비어 있지는 않은지 검사한다. 그다음 계정과 비밀번호가 맞는지 확인한다. 맞으면 issueToken 이 토큰을 만든다. 토큰은 이 사람은 로그인했다는 걸 증명하는 긴 문자열이다. 토큰 안에는 사용자 번호와 만료 시간만 넣고, 비밀번호는 넣지 않는다. 토큰은 서버의 비밀 키로 서명해서 만들고, 기본 90 초 동안만 쓸 수 있다.
출력: 성공하면 HTTP 200 과 함께 토큰, 유효 시간, 사용자 정보를 돌려준다.
실패 조건: 이메일이나 비밀번호 형식이 이상하면 400 BAD_INPUT. 비밀번호가 틀리거나 없는 계정이면 401 INVALID_CREDENTIALS. 이때는 토큰을 만들지 않는다. 비밀 키가 .env 에 없거나 형식이 틀리면 서버가 아예 켜지지 않는다.]

3 2 브 라 우 저 토 큰 저 장 과 전 송

파일 / 함수: [frontend-vue/src/api.js / login, authorizationHeaders]

코 드 발 췌

```
function authorizationHeaders() {
  // 토큰이 있을 때만 Authorization 헤더를 만든다 (없으면 헤더 없음)
```

```
  if (!session.accessToken) return {};
  return { Authorization: `Bearer ${session.accessToken}` };
}
export async function login(email, password) {
  // 서버에 로그인 요청을 보내고, 받은 토큰과 사용자를 메모리(session)에 저장한다
  const data = await request("/auth/login", {
    method: "POST",
    body: JSON.stringify({ email, password }),
  });
  session = { accessToken: data.accessToken, user: data.user };
  return data.user;
}
```

동작과 응답 코드 설명

[입력: 로그인 폼에 적은 이메일과 비밀번호
처리: login 이 서버에 로그인 요청을 보낸다. 성공해서 받은 토큰과 사용자 정보를 session 이라는 변수에 저장한다. 이 변수는 브라우저 메모리에만 있다. 그래서 새로고침하면 사라지고 다시 로그인해야 한다. 이후에 서버에 요청할 때는 authorizationHeaders 가 Authorization: Bearer 토큰 헤더를 붙여 준다. 토큰이 없으면 헤더를 만들지 않는다. 로그아웃 버튼을 누르면 clearSession 이 session 을 비운다.
출력: 로그인에 성공하면 사용자 정보를 돌려주고, 화면에 로그인한 사람의 이름이 보인다.
실패 조건: 비밀번호가 틀리면 401 INVALID_CREDENTIALS 이고 이메일 또는 비밀번호가 틀렸습니다가 나온다. 서버가 어떤 요청에서든 401 을 보내면 session 을 비워서 로그인 상태를 풀고, 다시 로그인하라는 안내를 보여 준다. 403(권한 없음)일 때는 로그인 상태를 풀지 않는다.

서 버 인 증 과 작 성 자 권 한

3 3 서 버 JWT 검 증

파일 / 함수: [backend/src/auth.js currentUserId]

코 드 발 췌

```
[ let data;
  try {
    data = jwt.verify(header[1], key, {
      algorithms: ["HS256"],
      issuer,
      audience,
    });
  } catch (e) {
    if (e instanceof jwt.TokenExpiredError)
      throw new ApiError(401, "TOKEN_EXPIRED", "토큰이 만료되었습니다.");
    throw new ApiError(401, "INVALID_TOKEN", "유효하지 않은 토큰입니다.");
  }

  // 3) 만료 시간(exp)이 없는 토큰은 거른다
  if (typeof data.exp !== "number")
    throw new ApiError(401, "INVALID_TOKEN", "유효하지 않은 토큰입니다.");

  // 4) 그 사용자가 실제로 있는지 확인한다
  const found = findPublic(Number(data.sub));
  if (!found)
    throw new ApiError(401, "INVALID_TOKEN", "존재하지 않는 사용자입니다.");
  return found.id;
```

동 작 과 응 답 코 드 설 명

[401 TOKEN_REQUIRED, INVALID_TOKEN, TOKEN_EXPIRED]

3 4 작 성 자 권 한 검 사

파일 / 함수: [backend/src/auth.js assertOwner, backend/src/app.js PUT /api/posts/:id]

코 드 발 췌

```
// BE-03: 로그인한 사용자와 글 작성자가 다르면 403 으로 막는다.
export function assertOwner(userId, authorId) {
  if (userId !== authorId)
    throw new ApiError(403, "FORBIDDEN", "본인이 쓴 글만 수정·삭제할 수 있습니다.");
}


app.put("/api/posts/:id", (req, res) => {
  const userId = currentUserId(req),
```

```
    id = postId(req),
    p = getPost(id);
  assertOwner(userId, p.authorId);
  res.json(store.update(id, fields(req.body, { title: 100, content: 5000 })));
});
```

동 작 과 응 답 코 드 설 명

[assertOwner 는 지금 요청한 사람(userId)과 글을 쓴 사람(authorId)이 같은지 비교한다. 다르면 403 FORBIDDEN 을 보낸다.userId 는 토큰을 검사해서 나온 값이고, authorId 는 DB 에 저장된 글의 작성자다. 토큰이 없거나 틀리면 401 이 나온다. 그래서 401 은 누군지 확인 못 할때, 403 은 누군지는 알지만 내 글이 아닐 때. authorId 를 보내면 400 BAD_INPUT 으로 거부하여 남의 이름으로는 글을 쓸 수 없다.성공하면 수정은 200, 삭제는 204 이다. 수정해도 작성자와 createdAt 은 그대로고 updatedAt 만 바뀐다. 없는 글은 404 NOT_FOUND 다.]

테 스 트 결 과

T01 부터 T17 까지 수행하고, 실제 응답과 증거의 위치를 기록하세요.

실행 환경: [Windows 11 Pro, Node.js v24.19.0, Microsoft Edg, 실행 명령 npm start(backend), npm run preview(frontend)]

로컬 배포 확인: [새 폴더에서 설치한 절차 / 프론트 빌드 결과 / 백엔드 실행 명령 / 4173 번 접속 및 API 연결 증거]

테스트 일시 및 코드 버전: [2026-10-28]

토큰 TTL: [일반 테스트 90 초 / 만료 테스트 3 초]

05_테스트시나리오.md 의 T01~T17 결과를 모두 기록하세요. PASS, FAIL, 미실행을 구분합니다. 예상 결과를 실제 결과로 복사하지 말고 직접 관찰한 HTTP 상태와 JSON code, 화면을 적으세요. 실제 결과 칸에는 예를 들어 403 / FORBIDDEN 처럼 기록합니다. 정상 응답은 200 / 해당 없음처럼 적습니다.

ID	수행 절차와 사용자	예상 결과	실제 HTTP 와 code	판정	증거
T01	[로그인하지 않은 상태에서 글 목록 조회]	[200, 목록과 상세가 보인다]	[200/해당없음]	[PASS]	[study-log-board\docs\evidence]
T02	[사용자 로그인 폼 제출]	[200]	[200/해당없음]	[PASS]	[study-log-board\docs\evidence]
T03	[잘못된 비밀번호 및 없는 이메일로 각각 로그인]	[401 INVALID_CREDENTIALS]	[401]	[PASS]	[study-log-board\docs\evidence]
T04	[로그인하지 않은 상태에서 내 정보 조회와 글 등록을 한다.]	[401]	[401]	[PASS]	[study-log-board\docs\evidence]
T05	[Alice 로 로그인해서 글을 쓰고 내 정보를 조회한다]	[200]	[200]	[PASS]	[study-log-board\docs\evidence]
T06	[Alice 로 자기 글을 수정하고 삭제한다]	[200,204]	[200,204]	[PASS]	[study-log-board\docs\evidence]
T07	[Bob 의 정상 토큰으로 Alice 가 쓴 글을 수정 요청한다.]	[403]	[403]	[PASS]	[study-log-board\docs\evidence]
T08	[Bob 의 정상 토큰으로 Alice 가 쓴 글을 삭제 요청한다.]	[403]	[403]	[PASS]	[study-log-board\docs\evidence]

ID	수행 절차와 사용자	예상 결과	실제 HTTP 와 code	판정	증거
T09	[정상 토큰의 서명 부분 첫 글자를 바꿔서 내 정보 조회, 글 등록, 수정, 삭제를 요청한다.]	[401]	[401]	[PASS]	[study-log-board\docs\evidence]
T10	[TTL 을 3 초로 서버를 다시 켜고 로그인한 뒤 4 초 이상 기다렸다가 내 정보 조회와 글 변경을 요청한다.]	[401]	[401]	[PASS]	[study-log-board\docs\evidence]
T11	[로그인한 뒤 로그아웃하고 내 정보 조회를 한다.]	[401]	[401]	[PASS]	[study-log-board\docs\evidence]
T12	[Bob 으로 로그인해서 글 등록 요청에 authorId:1 을 넣어 보내고, 따로 userId:1 을 넣어서도 보낸다.]	[400]	[400]	[PASS]	[study-log-board\docs\evidence]
T13	[내 토큰으로 빈 제목, 공백만 있는 내용, 너무 긴 입력을 각각 보낸다.]	[400]	[400]	[PASS]	[study-log-board\docs\evidence]
T14	[없는 글 번호(9999)로 조회, 수정, 삭제를 하고, 잘못된 번호(abc)로 조회한다]	[404]	[404]	[PASS]	[study-log-board\docs\evidence]
T15	[글을 쓰고 서버를 끈 뒤 다시 켜서 목록을 본다.]	[글, 작성자, 내용이 그대로 남아 있다.]	[예상과 같다]	[PASS]	[study-log-board\docs\evidence]
T16	[로그인한 뒤 페이지를 새로고침한다]	[글은 그대로 보이고, 로그인 상태는 풀린다.]	[예상과 같다]	[PASS]	[study-log-board\docs\evidence]
T17	[토큰 없이 수정과 삭제를 요청하고, Bob 으로 로그인해서 자기 글을 쓰고 수정하고 삭제한다]	[토큰 없는 요청은 401, Bob 의 본인 글 작업은 성공한다]	[예상과 같다]	[PASS]	[study-log-board\docs\evidence]

HTTP 응 답 확 인 기 록

예상 코드와 실제 코드를 대조하고, 201·204·401 의 헤더와 본문도 확인하세요.

숫자는 Network 나 API 클라이언트에서 직접 관찰한 값을 기록합니다. 본문에 있는 숫자만으로 판단하지 않습니다. 한 항목이 여러 요청을 포함하면 행을 추가하세요.

확인 항목	기대 응답	실제 HTTP 와 code	증거 위치
정상 조회·로그인·수정	**200**	[login 200, posts 200]	[T02.png, T01.png]
새 글 등록	**201 및 Location**	[201]	[기록]
본인 글 삭제	**204 및 빈 본문**	[204]	[기록]
입력 오류·추가 작성자 필드	**400 BAD_INPUT**	[400 / BAD_INPUT (T13)]	[T13.png]
잘못된 계정	**401 INVALID_CREDENTIALS**	[401]	[기록]
토큰 없음·헤더 형식 오류	**401 TOKEN_REQUIRED**	[401]	[기록]
변조·클레임 오류	**401 INVALID_TOKEN**	[401]	[기록]
정상 서명의 만료 토큰	**401 TOKEN_EXPIRED**	[401 / TOKEN_EXPIRED (T10)]	[T10.png]
다른 작성자의 변경	**403 FORBIDDEN**	[403 / FORBIDDEN (T07)]	[T14.png]
없는 글	**404 NOT_FOUND**	[404]	[기록]

[만료된 토큰으로 /me, 글 등록, 수정, 삭제를 요청했을 때 4 개 모두 401, TOKEN_EXPIRED 와 함께 이 헤더가 나왔다. 증거: T1.png]

[401 과 403 에서 화면이 각각 어떻게 달라지는지 설명]

[잘못된 입력(빈 제목, 공백 내용, 너무 긴 제목·내용)으로 등록을 요청한 뒤 공개 조회를 했더니 글은 바뀌지 않았다. 증거: T13.png]

[상 토큰으로 없는 글 수정은 T14 에서 404 NOT_FOUND 로 나왔다.]

테 스 트 증 거

캡처에는 테스트 ID 와 실제 응답을 표시하세요. 토큰 원문과 서버 비밀 키는 가립니다.

로 그 인 과 토 큰 전 송

[증거 1: 로그인 성공과 토큰이 첨부된 요청. 토큰 원문은 가리세요.]

[T02: Alice 로 로그인했더니 login 요청이 200 으로 나왔고, 화면에 "로그인했습니다"와 "Alice 님 로그인 중"이 표시되었다.T05: 로그인한 뒤 "내 정보 조회"를 누르니 me 요청이 200 으로 나왔고, Alice 의 정보가 나왔다.]

다 른 작 성 자 의 수 정 과 삭 제

[증거 2: Bob 이 Alice 의 글 수정·삭제를 시도한 서버 응답]

[T07: Bob 의 토큰으로 Alice 가 쓴 글을 수정하려고 했더니 서버가 403 Forbidden 을 보냈다.T08: Bob 의 토큰으로 Alice 가 쓴 글을 삭제하려고 했더니 서버가 403 Forbidden 을 보냈다.]

토 큰 변 조 와 만 료

[증거 3: 토큰 변조·만료 시 서버 응답과 화면 처리]

[T09: 토큰의 서명 첫 글자를 바꿔서 내 정보 조회, 글 등록, 수정, 삭제를 요청했더니 4 개 모두 401 INVALID_TOKEN 이 나왔다.T10: 토큰 유효 시간을 3 초로 하고 기다린 뒤 요청했더니 4 개 모두 401 TOKEN_EXPIRED 가 나왔고, 응답에 WWW-Authenticate: Bearer 가 있었다. 화면에서는 로그인이 풀리고 "로그인 시간이 지났습니다. 다시 로그인하세요."가 표시되었다.]

자 동 검 사 와 로 컬 배 포 증 거

[자동 검사 결과를 첨부했다면 실행 명령과 의미를 설명]

[T15: 서버를 껐다가 npm start 로 다시 켠 뒤에도 글 목록이 그대로였다. 글 번호, 작성자, 제목, 내용이 재시작 전과 같았다.]

AI 활 용 과 참 고 자 료

사용 범위, 실제 반영 위치와 직접 확인한 내용을 구분해 기록하세요.

AI 사용 여부: [사용]

사용하지 않았다면 미사용이라고 명시하고 참고한 수업 자료·공식 문서의 출처를 작성하세요. 사용했다면 전체 사용 범위를 요약하고 대표 사례 2~3 건을 작성하세요. 사용이 1 건이면 그 1 건만 적습니다. 대화 전체 제출은 필요하지 않습니다.

도구와 모델	사용 목적	반영한 파일 또는 보고서 항목	반영 방식
[Claude Code (Sonnet 5.5)]	[코드 작성, 오류 확인, 요청 시험]	[backend/src/auth.js (설정 읽기, issueToken, currentUserId, assertOwner), backend/src/app.js (두 줄)]	[수정 후 반영 (일부 코드는 빼도록 요청함)]

대 표 사 례

도구 / 날짜: [Claude Code (Sonnet 5.5)]/ 2026-10-28]

요청한 내용: [요청 헤더의 토큰을 확인해서 사용자 번호를 돌려주는 함수]

AI 가 제안한 내용: [헤더 모양 확인, jwt.verify 로 서명·만료·발급자·대상 확인, exp 와 sub 확인, 실제 있는 사용자인지 확인]

실제 반영 위치: [backend/src/auth.js currentUserId]

내가 확인하고 수정한 내용: [sub 모양을 따로 검사하는 줄이 꼭 필요한지 물었다. AI 가 확인해 보니 sub 가 이상해도 뒤의 사용자 확인에서 걸러져서 그 줄을 지웠다. 반면 exp 가 없는 토큰은 jwt.verify 만으로는 통과해서, exp 확인은 남겼다.]

검증 방법과 결과: [exp 없는 토큰, sub 없는 토큰, 서명 바꾼 토큰, 만료된 토큰으로 요청했을 때 각각 401 INVALID_TOKEN 또는 TOKEN_EXPIRED 가 나왔다]

참 고 자 료

자료 제목과 URL	사용한 위치	반영 내용 및 출처 표시
[수업에서 받은 과제 공지(01_과제공지.pdf)와 제공 템플릿, 안내 문서]	[전체]	[제공 코드 유지 부분, T01~T17 시험 절차를 따랐다.]

이 해 도 확 인 과 회 고

각 질문에 본인 코드의 동작을 근거로 답하세요.

1. 요청 본문의 authorId 를 믿으면 어떤 문제가 생기며, 내 코드는 어떻게 막는가?

[요청에 들어 있는 authorId 를 그대로 믿으면 누구나 다른 사람 이름으로 글을 쓸 수 있다. 예를 들어 Bob 이 authorId 를 1 로 적어 보내면 Alice 가 쓴 글처럼 저장된다. 내 코드는 이걸 두 가지로 막는다. 첫째, 글 등록과 수정은 title 과 content 만 받고, 다른 필드)가 있으면 field 가 거부한다. 둘째, 작성자는 요청에서 받지 않고, 서버가 토큰을 확인해서 얻은 사용자 번호로 직접 저장한다.]

2. 수정 버튼을 숨기는 것만으로 충분하지 않은 이유는 무엇이며, 서버의 어떤 함수가 권한을 검사하는가?

[버튼을 숨기는 건 화면만 바꾸는 것이라서, 누가 직접 요청을 보내면그대로 통과한다. 그래서 서버에서도 막아야 한다. 서버의 assertOwner 함수가 권한을 검사한다.]

3. JWT decode 와 verify 의 차이는 무엇이며, 내 코드는 어떤 검증을 수행하는가?

[decode 는 토큰 안의 내용을 읽기만 하고, 그 토큰이 진짜인지는 확인하지 않는다. verify 는 서명이 맞는지, 만료되지 않았는지 등을 확인해서 위조된 토큰을 걸러 준다. 내 코드는 jwt.verify 를 쓴다. 허용하는 서명 방식은 HS256 하나로 고정했고, 발급자와 대상도 확인한다. verify 는 만료 시간이 아예 없는 토큰도 통과시키기 때문에, exp 가 있는지는 따로 확인한다.]

4. 로그아웃, 토큰 만료, 새로고침은 각각 무엇을 바꾸는가? 이미 복사된 토큰은 어떻게 되는가?

[로그아웃은 화면이 가진 토큰과 사용자 정보(session)를 지워서, 다음 요청부터 Authorization 헤더가 붙지 않게 한다. 토큰이 만료되면(기본 90 초) 서버가 401 TOKEN_EXPIRED 를 보내고, 화면은 이때도 session 을 지우고 다시 로그인하라는 안내를 보여 준다. 새로고침하면 토큰이 브라우저 메모리에만 있어서 사라지고 로그인이 풀린다.]

5. 가장 어려웠던 오류 한 가지와 원인 확인·수정·재검증 과정을 설명하라.

[백엔드를 npm start 로 켰더니 서버 시작 실패: listen EADDRINUSE: address already in use 127.0.0.1:8000 오류가 났다. 8000 번 포트를 이미 다른 프로그램이 쓰고 있다는 뜻이었다. 원인은 이전에 켜 둔 서버가 꺼지지 않고 남아 있는 것이었다. 그 서버를 PowerShell 명령으로 종료한 뒤 npm start 를 다시 실행했더니 게시판 시작 코드 http://127.0.0.1:8000 메시지가 나오며 서버가 켜졌다. 이후 브라우저에서 4173 번 주소로 접속해 화면이 뜨는 것까지 확인했다.]

현재 남은 문제와 다음에 개선할 점: [없으면 없음으로 표시]

제출 확인: [ ] 실제 코드와 설명 일치 [ ] 테스트 직접 수행 [ ] AI 활용 및 출처 기록 [ ] 비밀 키와 토큰 제거

GitHub 확인: [ ] 루트 README 작성 [ ] 보고서 PDF 와 증거 업로드 [ ] README 문서 링크 확인 [ ] 교수자 접근 가능 [ ] 저장소 URL 과 커밋 SHA 제출